mock data prepare

FoodLens: Architecture and Workflow Documentation

standout features
1) smart meal: Using GenAI
2)Group Order: Shared Cart, Split share
3) Sustainability

Version 1.0 · MVP running on synthetic data · 12 October 2026

1. Purpose and scope

FoodLens is a cross-provider food comparison platform for the Indian delivery market. It does not just list restaurants. It reconciles fragmented data from several delivery providers into one trusted view of each restaurant, its menu, its prices, its availability and its reputation, then helps a user decide where and how to order.

This document describes how the MVP is built and why: the layered architecture, the module boundaries, the data model, every workflow step and how it is achieved in code, the formulas behind scoring and ranking, the API, the frontend, testing, and how to extend the system toward real provider integrations.

In scope: the working MVP in the `foodlens` repository (Node.js and Express backend, React and Vite frontend, synthetic provider adapters). Out of scope: real provider integrations, user accounts, payments, and trained ML models. These are listed in section 14 as extension points.

All provider names (FoodHub, QuickBite, DineDash), prices, ratings and reviews in the MVP are synthetic. Nothing is scraped and nothing is ordered.

2. Design principles

1. Provenance over pretence. A single source of truth does not mean pretending providers agree. It means one internal representation of identity, with traceable source-specific evidence and explicit uncertainty.
2. Unknown is not zero. A missing fee, ETA or rating is stored as `null`, shown as unknown, and scored neutrally. It is never silently converted to 0.
3. Never merge on a guess. Uncertain restaurant matches stay separate until a human approves them.
4. Estimates are labelled as estimates. Every total says whether it is an estimate, a lower bound or (in future) a verified checkout total.

5. Explainable ranking. Every ranked option carries its reasons and the weight, value and contribution of each factor.
6. Heuristics are labelled as heuristics. Reputation scores come from a word-list baseline and are never presented as verified facts.
7. Modular monolith. One deployable backend with clear folder boundaries. Services can be split out later without a rewrite.

3. System architecture

The system has three logical layers plus a provider-adapter boundary. Data flows upward from adapters to the UI. Requests flow downward from the UI to the data.

Layer	Folder	Responsibility
Provider adapters	`backend/src/data`	Return raw listings from each provider behind one interface
Data truth layer	`backend/src/pipeline`	Validate, resolve identity, match menus, reconcile attributes, store canonical data
Intelligence layer	`backend/src/intelligence`	Reputation, pricing, ranking and explanations
Application layer	`backend/src/server.ts`, `frontend/src`	REST API and React user interface

3.1 Repository structure

```
foodlens/
  backend/
    db/schema.sql              PostgreSQL target schema
    src/
      types.ts                 Raw and validated listing types, Adapter
interface
      data/synthetic.ts        Three synthetic provider adapters
      pipeline/
        text.ts                Normalisation, similarity, distance helpers
        validate.ts            Record validation
        resolve.ts             Entity resolution and review candidates
```

```
        menu.ts                 Cross-provider menu matching
        reconcile.ts            Canonical restaurant, observations, conflicts
        store.ts                In-memory canonical store
      intelligence/
        reputation.ts           Review analysis and rating aggregation
        pricing.ts              Basket pricing
        ranking.ts              Weighted ranking, badges, explanations
      server.ts                 Express routes
      tests/pipeline.test.ts    Eight unit and integration tests
  frontend/
    vite.config.ts              Dev proxy from /api to port 4000
    src/
      api.ts, types.ts, App.tsx, styles.css
      pages/                    Search, Restaurant, Admin
      components/               ProviderCard, ReputationPanel, SourcesPanel,
FreshPill
```

3.2 Provider adapters

Every provider sits behind the `Adapter` interface: a `provider` record (id, name, trust weight, kind) and an asynchronous `fetch()` returning raw listings. The raw shape is deliberately loose: fees, ETA and rating may be `null`, and nothing is assumed clean.

The three synthetic adapters return fixed data that is built to exercise every behaviour: three spellings of one restaurant, a different branch of the same chain, a similarly named but different restaurant, an uncertain match, a record with invalid coordinates, an item with a negative price, an unknown platform fee, a stale provider, and a syndicated duplicate review.

A real integration (an official API, a partner feed or an ONDC adapter) implements the same interface and is added to `new Store([...])`. Nothing else in the system changes.

3.3 Data truth layer

The pipeline converts raw listings into canonical restaurants. It has five stages, described in detail in section 6: validation, entity resolution, menu matching, reconciliation, and storage. The `Store` class orchestrates them.

3.4 Intelligence layer

The intelligence layer reads only validated canonical data. It computes three things: a reputation profile per restaurant, an estimated total per provider for a given basket, and a ranked, explained list of provider options. Formulas are in section 9.

3.5 Application layer

Express exposes REST endpoints under `/api`. The React frontend holds UI state (basket, priority, delivery limit) and calls the API when it changes. It contains no business logic. In development, Vite proxies `/api` to the backend, so the browser talks to a single origin.

4. Runtime topology

1. The backend starts and calls `store.init()`. The adapters are fetched in parallel with `Promise.allSettled`, so one failing provider is logged as an issue and does not stop the others.
2. Raw listings pass through validation, resolution and reconciliation. The canonical restaurants are held in memory.
3. The server begins listening on port 4000.
4. The browser loads the React app from the Vite dev server on port 5173. Calls to `/api/*` are proxied to port 4000.
5. Search, restaurant detail and compare requests are answered from the in-memory canonical data. Reputation, pricing and ranking are computed on demand, which is cheap at MVP scale.

5. Data model

5.1 In-memory shapes

- `RawListing`: what an adapter returns. Provider id, record id, name, address, coordinates, optional phone, cuisines, ETA, delivery fee, platform fee, rating and rating count, menu, optional offer, reviews, fetch timestamp and URL.
- `Listing`: a validated raw listing with a stable id of the form `providerId:recordId`.
- `CanonicalRestaurant`: one physical outlet, with name, area, median location, voted phone, merged cuisines, its provider listings, its matched menu items, its observations and its conflicts.

- `CanonicalItem`: one dish across providers, with a list of provider-specific offers (price, availability, the provider's own item name).
- `Observation`: attribute, value, provider, record id, observed time, confidence and validation status.
- `Conflict`: attribute, severity (info or warn), a description and the policy used to handle it.

5.2 PostgreSQL target

`backend/db/schema.sql` is the production schema. Key design decisions:

- `restaurants` holds canonical identity. `provider_listings` holds provider-specific records and maps each to at most one restaurant. A unique constraint on `(provider_id, provider_record_id)` prevents duplicate ingestion, and a unique constraint on `(restaurant_id, provider_id)` prevents a provider listing the same outlet twice.
- `observations` is append-only. Every price, fee, ETA and rating seen is a row with a timestamp, confidence and validation status. The latest value is found with an index on `(listing_id, attribute, observed_at DESC)`. Older rows are the price and reputation history.
- `reviews` stores a text hash so duplicate reviews across providers are found cheaply. `review_aspects` stores per-aspect sentiment with a model version.
- `entity_match_candidates` and `reconciliation_events` record the review queue and the policy decisions, so every merge is auditable.
- `user_preferences` carries an explicit analytics consent flag.

6. Workflow A: from raw listing to canonical data

This workflow runs at server start and again whenever a human resolves a review candidate.

Step 1: Fetch

How it is achieved: `Store.init()` calls `fetch()` on every adapter using `Promise.allSettled`. A rejected adapter adds an issue of the form adapter failed: provider unavailable, and the remaining providers continue.

Step 2: Validate

How it is achieved: `validate()` in `validate.ts` applies these rules to each raw listing.

- A missing name rejects the listing.
- Coordinates outside the valid range reject the listing.
- A rating outside 0 to 5 is set to unknown and an issue is logged.
- An ETA at or below 0, or 300 minutes or more, is set to unknown.
- A menu item with a non-positive price is dropped and logged. A `null` price is kept as unknown.

Every rejection is recorded as an issue with provider, record id, level and reason. The Data quality page lists them. Valid listings receive a stable id.

Step 3: Normalise

How it is achieved: `text.ts` lower-cases text, removes accents and punctuation, expands a few spelling variants (for example biriyani to biryani and rd to road), and splits into tokens. So Domino's, Dominos and M.G. Rd all reduce to comparable forms. The same helpers provide Levenshtein similarity, Jaccard and Dice overlap, containment, haversine distance and last-ten-digit phone extraction.

Step 4: Resolve identity

How it is achieved: `resolveEntities()` in `resolve.ts` processes listings in a deterministic order and compares each against the existing groups.

The pair score combines four signals:

- Name: the higher of the Levenshtein similarity of the normalised names and, when both have at least two tokens, the token containment.
- Geography: 1.0 within 100 metres, falling linearly to 0 at 500 metres.
- Address: Jaccard overlap of address tokens.
- Phone: 1 for an equal last-ten-digit number, 0 for a different number, absent when either side has no phone.

Without a phone the score is 0.40 name plus 0.35 geography plus 0.25 address. With a phone it is 0.30 name, 0.25 geography, 0.20 address and 0.25 phone.

Hard rules sit on top of the score. Listings more than 500 metres apart score 0 (a different branch). A phone mismatch caps the score at 0.84, so it can never merge automatically. A provider cannot appear twice in one group.

Decision thresholds: a best score of 0.85 or more merges into the existing group. A score from 0.60 up to 0.85 creates a review candidate and the listing stays separate. Anything lower creates a new restaurant. Human decisions override scores: a recorded merge scores 1 and a recorded rejection scores 0.

Outcome on the synthetic data: the three spellings of Biryani House in Vesu merge, the Adajan branch stays separate, Biryani Hub stays separate, and one Pizza Palace listing (no phone, pin about 280 metres away) lands in the review queue.

Step 5: Match menus

How it is achieved: `matchMenus()` in `menu.ts` works inside one canonical restaurant. Each item name is tokenised and noise words (full, regular, plate, dum) are removed. The word half marks a half portion. Two items match when their portion and vegetarian flag agree, the provider has not already contributed an item to that dish, and the Dice overlap of their core tokens is at least 0.75. So Chicken Dum Biryani (Full), Chicken Biryani and Chicken Biriyani Regular become one dish, while the half portion stays a separate dish.

Step 6: Reconcile attributes

How it is achieved: `buildRestaurant()` in `reconcile.ts` applies one policy per attribute.

Attribute	Policy
Name	Taken from the highest-trust provider, with the area suffix removed
Location	Median of provider coordinates
Phone	Majority vote, with trust as a tie-break
Cuisines	Union, case-insensitive
Price, fees, ETA, offers, ratings	Never merged. Kept per provider

Conflict detection runs alongside. Different phone numbers produce a warning. A provider pin more than 150 metres from the median produces a warning. Differing provider names and price spreads above 10 percent produce informational conflicts. Each conflict records the policy that handled it.

For every provider listing, observations are written for rating, ETA, delivery fee, platform fee, phone and each item price. Confidence is the provider trust weight multiplied by a freshness factor: 1.0 for fresh data, 0.8 for aging and 0.5 for stale.

Step 7: Store

How it is achieved: the `Store` keeps validated listings, issues, decisions, canonical restaurants and pending candidates. `rebuild()` recomputes the canonical view from listings and decisions. Because the pipeline is deterministic and cheap, a human decision is applied by recording it and re-running the whole pipeline. In production the same shapes live in PostgreSQL.

Freshness

Freshness is calculated at request time from the listing fetch timestamp: fresh up to 30 minutes, aging up to 180 minutes, stale beyond. It appears as a badge in the UI, lowers observation confidence, and applies a small ranking penalty.

7. Workflow B: a comparison request

This workflow runs each time the user changes the basket, the ranking priority or the delivery-time limit.

Step 1: The user action

The Restaurant page keeps the basket (item id to quantity), the priority and the maximum ETA in React state. A change triggers `POST /api/compare` with the restaurant id, the selections, the priority and the optional limit.

Step 2: Validation

The route checks that the restaurant exists, that selections are a non-empty array of an item id and an integer quantity from 1 to 20, that the priority is one of five known values, and that the limit is positive. Invalid input returns a 400 with a clear message.

Step 3: Reputation

The route calls `reputation()` for the restaurant to obtain per-provider delivery sentiment and per-source adjusted ratings (section 9.1).

Step 4: Pricing each provider

`priceBasket()` computes the estimated total for each listing (section 9.2). If a provider lacks any basket item, or lists it as unavailable, that provider is marked not available with the missing items named.

Step 5: Eligibility

`rank()` splits providers into eligible and excluded. A provider is excluded when it cannot supply the basket, or when its known ETA is above the user limit. An unknown ETA is not excluded. Each exclusion carries a reason that the UI shows.

Step 6: Scoring and ranking

Eligible options are scored using the weights for the chosen priority (section 9.3), sorted, and given badges, reasons and factor breakdowns.

Step 7: Response and rendering

The response contains the ranked options, the excluded options, the weights used and a disclaimer. React renders one provider card per option: rank, badges, total labelled estimate or at least, a line-by-line breakdown with unknown fees highlighted, the freshness badge, an expandable Why this rank section, and an outbound link labelled as a demo link.

8. Workflow C: human review of uncertain matches

1. The pipeline places uncertain pairs in `store.candidates` with the score and its parts: name, geography, address, phone and distance in metres.
2. `GET /api/admin/review-queue` returns the pending pairs with both listings' provider, name and address.
3. The Data quality page shows each pair with the evidence and two actions: merge, or keep separate.
4. `POST /api/admin/review-queue/:id/decision` records the decision and calls `rebuild()`.
5. In the rebuilt data the merged listings form one restaurant, or the rejection permanently keeps them apart. The queue entry disappears because the pair is no longer uncertain.

9. Intelligence formulas

9.1 Reputation (`reputation.ts`)

Method label: lexicon-baseline-v0. This is a heuristic baseline, not a validated model.

De-duplication. Review texts are normalised and compared across providers. Listings are ordered by trust, so the most trusted source keeps a duplicated review and the others are removed. The count removed is reported.

Aspect analysis. Each review is split into clauses at punctuation and at the words but and however. Each clause is scored using small positive and negative word lists, with a flip when a negator appears within the previous two words. A clause with a non-zero sentiment is tagged with every aspect whose keywords it contains: food, delivery, value or service. So the example The biryani was excellent, but the delivery took almost an hour yields positive food and negative delivery.

Aspect score. Each mention is weighted by recency: weight is 0.5 raised to (age in days divided by 90). The aspect score is the weighted sum of sentiments divided by (total weight plus 1), then mapped from the range minus 1 to 1 into 0 to 1. The plus 1 shrinks sparse evidence toward neutral. An aspect has enough evidence at 3 or more mentions.

Provider delivery. Delivery mentions are aggregated per provider. This keeps restaurant food quality separate from provider-specific delivery performance.

Rating aggregation. Each source rating is shrunk toward a prior: adjusted equals (n times raw plus 30 times 3.9) divided by (n plus 30), where n is the rating count. Sources are combined by weighted average with weight ln(1 plus n). A per-provider calibration offset is supported and defaults to 0. So 4.4 from 95 ratings does not outrank 4.3 from 1,240.

Food quality. With at least 3 food mentions it is 0.7 times the combined rating plus 0.3 times (1 plus 4 times the food score). Otherwise it is the combined rating alone, and the basis is stated.

Evidence level. High needs at least 500 total ratings and 10 analysed reviews. Medium needs at least 100 ratings. Otherwise low.

9.2 Pricing (`pricing.ts`)

- Subtotal is the sum of price times quantity for every selected item.
- Food GST is assumed at 5 percent of the subtotal. GST on fees is assumed at 18 percent of delivery plus platform fee. Both are stated as assumptions in every response.

- The discount is the lower of the offer cap and the offer percentage of the subtotal, applied only when the subtotal reaches the minimum order. When it does not, the card shows how much more to add.
- Total equals subtotal plus food GST plus delivery fee plus platform fee plus GST on fees minus discount.
- If a fee is unknown it adds 0 to the sum, the unknown fee is listed, and the total is flagged as a lower bound and shown as at least.
- The verified flag is always false in the MVP. It may become true only when an official integration confirms a checkout total.

9.3 Ranking (`ranking.ts`)

Four factors are scored from 0 to 1:

- Cost: min-max normalised across eligible options, lower is better.
- ETA: min-max normalised, lower is better. An unknown ETA scores 0.5.
- Reliability: the provider's delivery sentiment, used only with at least 3 delivery mentions. Otherwise 0.5.
- Rating: the provider's adjusted rating mapped from 1 to 5 onto 0 to 1.

The final score is the weighted sum minus a freshness penalty (0 fresh, 0.02 aging, 0.06 stale). The weights are hypotheses to be tested with users:

Priority	Cost	ETA	Reliability	Rating
Best value	0.40	0.20	0.25	0.15
Cheapest	0.70	0.10	0.10	0.10
Fastest	0.10	0.70	0.10	0.10
Most reliable	0.10	0.15	0.65	0.10
Best rated	0.10	0.10	0.20	0.60

Badges (Cheapest, Fastest, Best overall value, Most reliable, Best rated) are computed independently of the chosen priority. Most reliable needs at least 3 mentions and a reliability of at least 0.55. Best overall value is the winner under the Best value weights.

Limitations: min-max scaling exaggerates small differences when options are close, and food quality is restaurant-level, so it does not separate providers within one restaurant.

10. API reference

Method and path	Purpose
GET `/api/meta`	Data mode, synthetic notice, providers, priority presets
GET `/api/search`	Query parameters q, cuisine, maxEta, minRating, page, pageSize. Returns restaurant cards
GET `/api/restaurants/:id`	Canonical profile, provider listings, matched menu, reputation, conflicts, observations
POST `/api/compare`	Body: restaurantId, selections, priority, maxEta. Returns ranked options, exclusions, weights
GET `/api/admin/review-queue`	Pending uncertain matches
POST `/api/admin/review-queue/:id/decision`	Body: verdict merge or reject
GET `/api/admin/pipeline`	Counts and validation issues

Errors use one shape: an `error` object with a `code` (BAD_REQUEST or NOT_FOUND) and a `message`. Search is paginated with `page` and `pageSize` (maximum 50) and returns `total`.

Search relevance: every query token must match somewhere. A match in the name scores 3, in cuisines 2, in dish names 1. Results sort by relevance, then by combined rating. Dish matches return a price range across providers.

11. Frontend

- Search page: debounced search box (250 ms), cuisine chips, result cards with matched dish prices, provider ETAs and freshness badges, plus loading, empty and error states.
- Restaurant page with three tabs. Compare: basket stepper, five priority chips, an optional maximum delivery time, provider cards and exclusion notices. Reputation: ratings per source with adjusted values, aspect bars with a low-evidence marker,

delivery sentiment per provider, praised and criticised snippets, and the method disclaimer. Sources and freshness: provider listings, conflicts with the policy used, and the raw observation table.

- Data quality page: pipeline counts, validation issues and the review queue with merge and keep separate actions.
- The App shell shows a persistent synthetic-data banner and an API-unreachable message if the backend is down.
- Accessibility: visible focus rings, labelled controls, tab roles, reduced-motion support, and a responsive layout that stacks on narrow screens.

12. Flowcharts

Three flowcharts describe the system. Each is explained here in words and drawn in the diagrams that follow.

12.1 System structure

Provider adapters feed the data truth layer (validate, resolve, reconcile, store). The intelligence layer (reputation, pricing, ranking) reads the canonical store. The application layer (Express routes, Vite proxy, React) exposes the results. Requests travel down, data travels up.

12.2 Ingestion and entity resolution

A raw listing is validated. Invalid listings are rejected and logged. Valid listings are scored against known restaurants. A best score below 0.60 creates a new restaurant. A score from 0.60 up to 0.85 goes to the review queue, and a human approval merges it. A score of 0.85 or more merges automatically. All paths end in reconciliation and storage.

12.3 Compare request

The basket and priority arrive at the compare endpoint. Each provider is priced. Providers that cannot fulfil the basket or exceed the delivery limit are excluded with a reason. Eligible providers are scored, with unknown values treated as neutral. The options are ranked, explained, and rendered.

13. Testing

Run `npm test` in the backend. Eight tests cover the behaviours that matter most.

1. The three spellings of one restaurant merge into a single canonical restaurant.
2. A different branch and a similarly named restaurant are not merged.
3. An uncertain match goes to the review queue and is not merged.
4. Approving a candidate merges the restaurants and empties the queue.
5. Invalid coordinates and a negative price are rejected with reasons.
6. Menu matching handles spelling and size variants, and keeps the half portion separate.
7. Pricing flags an unknown fee as a lower bound and respects the offer threshold.
8. Aspect analysis splits a mixed review into positive food and negative delivery.

The backend type-checks with `npm run typecheck`. The frontend builds with `npm run build`, which type-checks first.

Future tests: integration tests against PostgreSQL, API contract tests for every route, and evaluation of the sentiment step on a labelled review sample before any accuracy is claimed.

14. Extending the system

Real providers. Do not rely on unrestricted public APIs or scraping. Pursue official partner APIs, authorised feeds, ONDC-compatible integrations and direct restaurant partnerships. Implement `Adapter`, set the provider trust weight, and register it in the Store. Respect licensing and data ownership. Only an integration that actually supports price or checkout verification may set the verified flag.

PostgreSQL. Apply `backend/db/schema.sql`, replace the in-memory Store with queries, and keep the pipeline functions as pure transformations. Write observations as append-only rows to give price and reputation history.

Background jobs. Move ingestion and reconciliation to scheduled jobs. Add Redis only after measuring that caching helps. Add a message broker only if the workload justifies it.

ML. Replace the word-list analysis with a pretrained sentiment and aspect model, but only after building a labelled evaluation set and comparing it to the baseline. Suspicious-review detection should be evidence-based anomaly detection and should never auto-label a customer as fraudulent.

Users and privacy. Add authentication, saved restaurants and preferences. Collect behavioural data only with explicit consent, as modelled by the analytics consent flag.

Production readiness. Rate limiting, structured logging, monitoring, data-quality dashboards, a security review, CI and deployment.

15. Known limitations

- All data is synthetic, so ratings, prices and availability say nothing about the real market.
- Sentiment is a small word-list heuristic. It misses sarcasm, mixed clauses and non-English text.
- GST rates are assumptions and discounts are modelled simply (percentage with a cap and a minimum order).
- Ranking weights are untested hypotheses. Min-max scaling can exaggerate small gaps.
- The Store is in memory, so decisions and data reset on restart.
- Provider data freshness is simulated relative to server start.
- There is no authentication, so the admin routes are open.
- Outbound links are demo links. Nothing is ordered.